KAIROS: Measuring What Caps a Reinforcement-Learning Agent That Drives Parametric CAD

Sunkalp Chandra
sunkalp.chandra@gmail.com

Abstract

We present KAIROS, an agent that builds parametric CAD models by emitting structured operations into a live FreeCAD kernel from a natural-language requirement, together with a benchmark that measures how far such an agent actually gets. Our central finding is methodological: on a task of this shape, *the headline score is usually measuring the harness rather than the policy*. We make that concrete. An oracle that replays the ground-truth expert through the agent's own action codec scores 0.594 progress and 0.500 full-build success — not 1.0 — so every learned number must be read against that ceiling rather than against perfection. Behavioural cloning reaches 0.983 held-out next-action accuracy yet only 0.445 progress in closed loop, and a PPO stage anchored to it reaches 0.413; a paired bootstrap over identical tasks puts their difference at +0.031 with a 95% interval of $[-0.008, +0.091]$ (5/1/26 win/loss/tie), so the benchmark *cannot separate them*. Ablations show the policy does read its requirement, and that much of its low invalid-action rate belongs to the action mask rather than to the policy. Along the way we report four defects that each produced a plausible, publishable, wrong number, and the audits that now prevent them.

1 Introduction

Computer-aided design is an attractive target for sequential decision making. A part is built by an ordered sequence of discrete, parameterised operations — sketch a profile, pad it, pocket a hole, fillet an edge — and the result is checkable: mass, wall thickness, hole count and angles are all measurable against a written requirement. That combination, a discrete action space with a programmatic reward, is rare outside of games.

It is also a setting where it is unusually easy to report a number that is not about the policy. A CAD agent sits behind several layers of machinery — an action codec, a geometry kernel, a constraint checker, a requirement parser — and each layer can fail *silently* and *plausibly*. The agent still runs, the kernel still returns valid solids, the checker still reports satisfaction, and the resulting score looks like a result.

This paper is organised around that hazard. We describe the system (Section 3), the benchmark (Section 4), and the results (Section 5). We then devote Section 6 to four defects we found in our own pipeline, each of which had already produced a number we believed. We think this is the most transferable part of the work.

Contributions.

1. A CAD environment exposing **38 structured operations** against a live FreeCAD kernel, with a JSON bridge that lets a PyTorch policy drive an interpreter that cannot import PyTorch.
2. **KAIROS-CAD-v1**, a benchmark of BUILD and COMPLETE(k) tasks over a frozen three-way split, scored by a prefix-ordered milestone ladder rather than by success alone.
3. An **oracle-replay baseline** that measures the ceiling the action codec imposes. We argue this belongs in any agent benchmark whose action space is a lossy encoding of the demonstrations.
4. A catalogue of **silent-failure modes** in evaluating such agents, each with the measurement that exposes it, all now automated.

2 Related work

Learning CAD construction. DeepCAD [12] and Computer-Aided Design as Language [4] model construction sequences generatively, and the Fusion 360 Gallery [11] supplies human design sequences with a reconstruction environment. These generate or reconstruct sequences; KAIROS instead executes actions against a live kernel and scores the resulting solid against a written requirement, which is what makes the harness itself a measurement problem.

Imitation and compounding error. Behavioural cloning [6] degrades under its own state distribution, the failure DAgger [7] addresses. Our COMPLETE(k) family measures that degradation directly rather than inferring it from a closed-loop gap: sweeping how many trailing actions the policy must supply turns compounding error into a curve. We fine-tune with PPO [9] using GAE [8] and a KL anchor back to the cloned policy.

Evaluation practice. Our statistical treatment follows the reproducibility literature in deep RL [1, 5], which documents how few-seed, few-episode comparisons produce differences that do not survive resampling. We use paired bootstrap intervals [3] because every policy faces identical tasks. The point we add is complementary and, we think, under-discussed: before asking whether two policies differ, one should ask what the harness would score *given a perfect policy*, because that number is frequently not 1.0.

3 System

3.1 Action space and environment

The environment wraps FreeCAD's [10] PartDesign workbench, exposed through a Gym-style step interface [2]. A policy emits a triple $(o, \mathbf{p}, t)$: an operation index over 38 operations, six continuous parameters in $[0, 1]^6$, and a target index selecting an edge, face or feature from the live document. A codec decodes this into a validated structured action; illegal actions are masked before sampling, and actions that pass the mask but fail in the kernel are returned as failures rather than exceptions, so the policy experiences them as negative outcomes rather than crashes.

Two interpreters. FreeCAD ships its own Python, which cannot install PyTorch. Rather than reimplement geometry, we split the system: the environment runs under FreeCAD's interpreter, the policy under the system interpreter, and they exchange newline-delimited JSON over a pipe. Everything in the environment package is import-clean under both.

3.2 Data

An expert procedural generator produces designs across eight parametric families (brackets, plates, flanges, spacers). Each design carries a natural-language requirement, the full action trajectory, the resulting solid, and measured properties. Requirements are generated *from* the sampled parameters, so ground truth is exact rather than annotated.

Crucially, expert trajectories are recorded as actions the codec can express. This is not automatic; Section 6 describes what happened when it was not true. The current dataset round-trips 16,452 expert steps through the codec with 0 unusable and a worst-case drift of 0.0005 mm, two orders of magnitude below the 0.1 mm tolerance every constraint check uses.

3.3 Policy and training

The policy is a 1.14M-parameter model mapping requirement text and current geometry to the next structured action, with separate heads for the operation, the parameters and the target. We train by behavioural cloning for 40 epochs, holding out by *design* rather than by step — a step-level split puts actions from the same trajectory on both sides and inflates accuracy. We then run PPO for 25 iterations with a clipped surrogate and a KL anchor to the cloned policy, on a dense milestone reward.

Policy	Progress	Success	Validity	Satisfaction	Efficiency
Oracle replay †	0.594	0.500	0.924	0.656	1.000
Behavioural cloning	0.445	0.281	0.957	0.453	0.620
PPO (BC-anchored)	0.413	0.250	0.951	0.477	0.675
Scripted from spec	0.224	0.000	1.000	0.211	0.640
Immediate finish	0.217	0.156	1.000	0.266	1.000
Legal random	0.146	0.000	0.676	0.268	0.604

Table 1: KAIROS-CAD-v1, held-out test split. †Oracle replay is not a policy: it replays the ground-truth expert through the action codec, so its score is the *ceiling*, and the gap between it and 1.000 measures what the codec and the target encoding cannot express.

4 Benchmark

4.1 Tasks

KAIROS-CAD-v1 defines two task types over a frozen 70/15/15 split by design:

- **BUILD**: given only the requirement, build the part from an empty document.
- **COMPLETE(k)**: given the requirement and the expert's trajectory with its last k actions removed, supply those k actions. Larger k is harder.

COMPLETE(k) exists because BUILD alone is close to all-or-nothing at this scale and reports 0.000 for policies that differ substantially. Sweeping k measures compounding error directly.

4.2 Scoring

Success rate alone cannot rank policies that all fail. We score a *prefix-ordered milestone ladder* with strictly dominating weights: opened a sketch, drew geometry, made a solid, solid is valid, has a hole, all constraints met, finished successfully. Credit stops at the first milestone missed. The prefix rule is what stops a constraint check passing *vacuously* on geometry that was never built and ranking an empty document above a real but imperfect part.

4.3 Baselines

Four baselines bound the task, and two of them audit the harness rather than compete:

- **Oracle replay** pushes the ground-truth expert actions through the *same codec* a policy uses. Its score is the ceiling.
- **Scripted from spec** builds the one shape a fixed script can build from the parsed requirement: a plate with holes. It should nail flat families and fail bent ones.
- **Immediate finish** terminates at once. Any policy below it is worse than doing nothing.
- **Legal random** samples uniformly from the masked-legal actions.

5 Results

5.1 Main results

Table 1 reports the leaderboard over the held-out test split.

Two things are worth stating plainly. First, the oracle does not score 1.000; it scores 0.594. Reading behavioural cloning's 0.445 against 1.0 overstates the shortfall by roughly a third of the scale. Second, behavioural cloning reaches 0.983 next-action accuracy on held-out designs while reaching 0.445 in closed loop. The gap is compounding error, and Table 2 measures it directly.

Policy	k=1	k=2	k=4	k=8	full build
Oracle replay †	0.833	0.600	0.333	0.500	0.375
Behavioural cloning	0.833	0.600	0.333	0.000	0.000
PPO (BC-anchored)	0.833	0.600	0.000	0.000	0.000
Immediate finish	0.833	0.000	0.000	0.000	0.000

Table 2: Success against difficulty. k is the number of trailing actions the policy must supply; the expert prefix is everything before it, so larger k is harder. Behavioural cloning holds up when one action remains and has collapsed by eight.

Comparison	Δ progress	95% CI	W/L/T	Separates
`legal-random vs. oracle-replay`	-0.447	$[-0.588, -0.310]$	2/27/3	yes
`immediate-finish vs. oracle-replay`	-0.376	$[-0.527, -0.235]$	1/25/6	yes
`oracle-replay vs. scripted-spec`	+0.369	$[+0.234, +0.507]$	22/9/1	yes
`bc vs. legal-random`	+0.298	$[+0.204, +0.402]$	27/2/3	yes
`legal-random vs. ppo`	-0.267	$[-0.371, -0.174]$	2/26/4	yes
`bc vs. immediate-finish`	+0.228	$[+0.141, +0.329]$	22/1/9	yes
`bc vs. scripted-spec`	+0.220	$[+0.126, +0.323]$	21/2/9	yes
`immediate-finish vs. ppo`	-0.196	$[-0.293, -0.115]$	1/22/9	yes

Table 3: Paired bootstrap on the per-task difference in progress score, 5,000 resamples. An interval straddling zero means this benchmark cannot separate the two policies at this sample size.

5.2 The comparison that does not hold

Behavioural cloning appears to edge PPO, 0.445 against 0.413. Every policy faces identical tasks in identical order, which makes the paired bootstrap the correct test: most of the variance at this sample size is *between tasks*, and pairing removes exactly that.

The result (Table 3) is that **BC and PPO do not separate**: +0.031 with a 95% interval of $[-0.008, +0.091]$, and 5/1/26 in wins, losses and ties. Reporting "BC beats PPO" from the point estimates would repeat an error we had already made once, when a six-episode evaluation reported 0.500 for a policy that scored 0.286 over fourteen.

5.3 What the policy is actually using

Table 4 separates two claims that are easy to conflate. Shuffling the requirement — pairing a design with a different design's text — costs the policy substantially, so it is reading the requirement rather than memorising a family prior. But removing the action mask collapses validity, which means a large share of the policy's low invalid-action rate belongs to the mask and not to the policy. Both facts follow from the same experiment and only one of them is flattering.

6 Four ways to measure the harness instead of the policy

Each of the following produced a number we believed and reported. Each is now covered by an automated audit.

1. An action space that cannot express its own demonstrations. Six of eight families sketched their outline as a single polygon carrying an arbitrary vertex list. The codec can only express a polygon as a *regular* n-gon, so those steps were unrepresentable: cloning silently dropped them (7.81% of all expert steps, touching 829 of 1,080 designs) and an oracle replaying the expert could not rebuild the part. The measured "ceiling" was an artefact of the encoding, not a property of the task. Expanding each profile into one line segment per edge drove unrepresentable steps to zero.

2. Normalisation that clips instead of failing. The codec maps each parameter into a fixed range. Values outside that range were *clipped*. A clipped value is not a rounding error; it is a different

Condition	Progress	Δ	Validity
Behavioural cloning (intact)	0.445	—	0.957
requirement shuffled	0.343	-22.9%	0.955
requirement blanked	0.374	-15.9%	1.000
action mask removed	0.339	-23.9%	0.407

Table 4: Ablations. Each row corrupts one input and re-runs the identical suite.

action. A sketch offset of 89.9 mm against a ± 50 mm range decoded back as 50 mm, so the oracle built the feature 40 mm from where the expert put it — with every step reporting success and zero invalid actions. This silently corrupted 260 expert steps across 206 of 1,080 designs (19.1%) while every audit still called the trajectory fully representable. Encoding now *raises* on an out-of-range value, and ranges are named once and shared by both directions; they had drifted apart, so an in-range pad length round-tripped 61.9 mm away having raised nothing at all.

3. A requirement the ground truth cannot satisfy. The constraint checker reads bounds from the requirement *text*, not from the float the generator used. Rounding a minimum to nearest states a bound the part can miss: a 6.1866 mm wall written as "6.2 mm" makes the expert violate its own requirement while having built exactly the right part. A second cause compounded it — one family declared its minimum wall as the plate thickness while its gusset rib, also a wall, could be thinner. Together these marked 139 of 1,080 designs (12.9%) as failures. Minimums now round down, maximums round up, and the stated bound is derived from the thinnest feature.

4. Model selection on the evaluation set. An early PPO run selected its best checkpoint on the same pool the evaluation scored, and reported 0.286 closed-loop success on requirements it had trained on. The frozen three-way split in Section 4 exists because of this. The corrected number was 0.000 — and the 0.000 was itself misleading for a different reason, being measured only on full builds, which is why COMPLETE(k) exists.

The common shape. None of these raised an exception. Each produced a plausible number, in the expected direction, of the expected magnitude. What caught them was not testing that the code runs but *measuring the harness against a known answer*: replaying ground truth through the agent's own machinery and asking why the score was not perfect. We would suggest that any benchmark whose action space is a lossy encoding of its demonstrations should report an oracle-replay ceiling as a matter of course.

7 Limitations

The learned results here are weak in absolute terms, and we state the ceiling rather than the excuse. Full builds succeed rarely; the benchmark separates BC and PPO from the null baselines but not from each other. Target selection — choosing which edge or face an operation applies to — is not recoverable by the current encoder, which caps families whose recipes pattern a specific feature. The RL stage is single-seed. Families are procedural, so requirement language is templated rather than natural. Wall thickness is measured by ray casting, which gives an upper bound: failing it is conclusive, passing it is necessary but not sufficient.

8 Conclusion

We built a CAD agent and a benchmark, and the most durable result was neither the agent nor the score but the discipline of measuring the measuring apparatus. Four separate defects in our pipeline each produced a confident wrong number, and in every case the thing that exposed it was replaying a known-correct trajectory through the agent's own machinery and refusing to accept a score below perfect without an explanation. We release the environment, the dataset generator, the frozen benchmark, and the audits.

Reproducibility

Code, dataset generator, frozen splits, and every audit in this paper are at https://github.com/sunkalpchandra/kairos-cad. Every number in every table is generated from committed run artifacts by `scripts/build_paper_tables.py`; none is typed by hand. The dataset is regenerated deterministically from seeds by `make generate-data`.

References

[1] Rishabh Agarwal, Max Schwarzer, Pablo Samuel Castro, Aaron Courville, and Marc G. Bellemare. Deep reinforcement learning at the edge of the statistical precipice. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2021.

[2] Greg Brockman, Vicki Cheung, Ludwig Pettersson, Jonas Schneider, John Schulman, Jie Tang, and Wojciech Zaremba. OpenAI gym. *arXiv preprint arXiv:1606.01540*, 2016.

[3] Bradley Efron and Robert J. Tibshirani. *An Introduction to the Bootstrap*. Chapman & Hall/CRC, 1993.

[4] Yaroslav Ganin, Sergey Bartunov, Yujia Li, Ethan Keller, and Stefano Saliceti. Computer-aided design as language. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2021.

[5] Peter Henderson, Riashat Islam, Philip Bachman, Joelle Pineau, Doina Precup, and David Meger. Deep reinforcement learning that matters. In *AAAI Conference on Artificial Intelligence*, 2018.

[6] Dean A. Pomerleau. ALVINN: An autonomous land vehicle in a neural network. In *Advances in Neural Information Processing Systems (NeurIPS)*, 1988.

[7] Stéphane Ross, Geoffrey J. Gordon, and J. Andrew Bagnell. A reduction of imitation learning and structured prediction to no-regret online learning. In *International Conference on Artificial Intelligence and Statistics (AISTATS)*, 2011.

[8] John Schulman, Philipp Moritz, Sergey Levine, Michael I. Jordan, and Pieter Abbeel. High-dimensional continuous control using generalized advantage estimation. In *International Conference on Learning Representations (ICLR)*, 2016.

[9] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.

[10] The FreeCAD Team. FreeCAD: An open-source parametric 3d CAD modeler. https://www.freecad.org, 2026. Version 1.1.3.

[11] Karl D.D. Willis, Yewen Pu, Jieliang Luo, Hang Chu, Tao Du, Joseph G. Lambourne, Armando Solar-Lezama, and Wojciech Matusik. Fusion 360 gallery: A dataset and environment for programmatic CAD construction from human design sequences. In *ACM Transactions on Graphics (SIGGRAPH)*, 2021.

[12] Rundi Wu, Chang Xiao, and Changxi Zheng. DeepCAD: A deep generative network for computer-aided design models. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, 2021.